

UNIVERSIDAD NACIONAL DE COLOMBIA
SEDE BOGOTÁ D.C.

Verificación de sistemas digitales

Practica N°1: Metodología de Diseño Tradicional

Autores:
Santiago Herrera Acuna,
Juan Diego Vasquez Quintero,
David Ricardo Villarreal Archila.
Correos electrónicos:
saherreraa@unal.edu.co,
juvasquezq@unal.edu.co,
dvillarreal@unal.edu.co.

Facultad de Ingeniería.
Departamento de Ingeniería Eléctrica y Electrónica.

Índice

1. Resumen **3**

2. Objetivos de aprendizaje **3**

3. Planteamiento inicial **3**

4. Camino de datos **4**
4.1. Restador 6
4.2. Restador de 8 bits 6
4.3. Registro de 8 bits 7
4.4. Reducción OR de 8 bits y Multiplexor 9
4.5. Integración estructural del Camino de Datos 9

5. Máquina de estados de la unidad de control **10**

6. Resultados **15**
6.1. Integración del camino de datos y la unidad de control 15
6.2. Simulación 16
6.3. Circuito sintetizado 17
6.3.1. Submódulo `mcd_top` 19
6.3.2. Submódulos `button_frontend` y `touch_frontend` 21
6.3.3. Submódulo `input_control` 23
6.4. Recursos utilizados 24

7. Anexos **25**

1. Resumen

En este informe de laboratorio se presenta el diseño e implementación digital del algoritmo para el cálculo del Máximo Común Divisor (GCD, por sus siglas en inglés). Para ello, se diseñó una ASM (*Algorithmic State Machine*) que permitió definir la arquitectura del camino de datos (**datapath**) y la unidad de control (**control unit**).

Posteriormente, ambos módulos fueron modelados en el lenguaje de descripción de hardware Verilog y se validó su correcto funcionamiento mediante simulación funcional con Icarus Verilog, utilizando Surfer para la inspección de las formas de onda. La verificación automatizada del núcleo incluyó 43 casos de prueba con diferentes combinaciones de operandos, incluyendo condiciones representativas y casos límite.

Finalmente, se realizó la síntesis y análisis del diseño mediante Intel Quartus y, adicionalmente, se implementó un flujo de herramientas abierto compuesto por Yosys para síntesis lógica, `nextpnr-ecp5` para *place and route*, `ecppack` para la generación del archivo de configuración y `openFPGALoader` para la programación de una FPGA Lattice ECP5 LFE5U-45F instalada en una tarjeta Colorlight i9. Se analizaron la arquitectura RTL obtenida, la utilización de recursos físicos, el cumplimiento temporal y el funcionamiento final del sistema sobre hardware real.

2. Objetivos de aprendizaje

- Fortalecer las metodologías de diseño e implementación de sistemas digitales jerárquicos (camino de datos y unidad de control).
- Identificar y analizar la estructura del circuito obtenido a partir del proceso de síntesis comportamental.
- Dimensionar y evaluar la utilización de recursos en la FPGA al implementar el sistema sintetizado.
- Desarrollar e implementar un entorno de pruebas automatizado (*testbench*) para la verificación del diseño mediante vectores de entrada.

3. Planteamiento inicial

Para la implementación del sistema digital que ejecuta el cálculo del GCD, se partió del siguiente algoritmo descrito en código de alto nivel:

```
while (X != Y) {
    if (X < Y)
        Y = Y - X;
    else
        X = X - Y;
}
```

A partir de este comportamiento, se diseñó una ASM (máquina de estados algorítmica de tipo Moore) que permitió derivar tanto el camino de datos como la unidad de control (Figura 1).

Para el **camino de datos**, el diagrama permitió determinar la necesidad inicial de:

- **Dos registros:** para almacenar los operandos X y Y.
- **Dos restadores:** para ejecutar las operaciones $Y = Y - X$ y $X = X - Y$.
- **Dos multiplexores:** para seleccionar entre los datos de entrada iniciales (A, B) o los valores realimentados de las restas.
- **Módulos comparadores/detectores:** para evaluar las condiciones $X \neq Y$ (`neq`), $X < Y$ (`lt`) y la presencia de ceros en las entradas (`zero`).

Respecto a la **unidad de control**, la carta ASM contempla un total de 6 estados codificados en 3 bits (Figura 1):

1. **S0 · IDLE (000):** Estado de reposo o inactividad. Permanece en este estado evaluando la señal de inicio (`start`).

2. **S1 · LOAD (001):** Tras recibir la señal `start=1`, se habilitan las señales de carga de los registros (`ldX=1`, `ldY=1`) y se seleccionan las entradas primarias A y B con `sel=0`.
3. **S2 · CHECK (010):** Estado de evaluación lógica donde se analiza si los datos son cero (`zero`), la desigualdad (`neq`) y la relación de orden (`lt`). La señal de operación, que se explicara en la sección posterior, se fija en `op=0`.
4. **S3 · SUBY (011):** Si `lt=1` ($X < Y$), se ejecuta la resta $Y \leftarrow Y - X$ actualizando el registro Y (`sel=1`, `ldY=1`, `op=1`).
5. **S4 · SUBX (100):** Si `lt=0` ($X \geq Y$), se ejecuta la resta $X \leftarrow X - Y$ actualizando el registro X (`sel=1`, `ldX=1`, `op=0`).
6. **S5 · DONE (101):** Cuando finaliza el cálculo (`zero=1` o `neq=0`), el sistema activa la señal `done=1` para indicar la finalización. El sistema retorna al estado **S0 · IDLE** únicamente cuando la señal externa `start` vuelve a 0 (`start=0`).

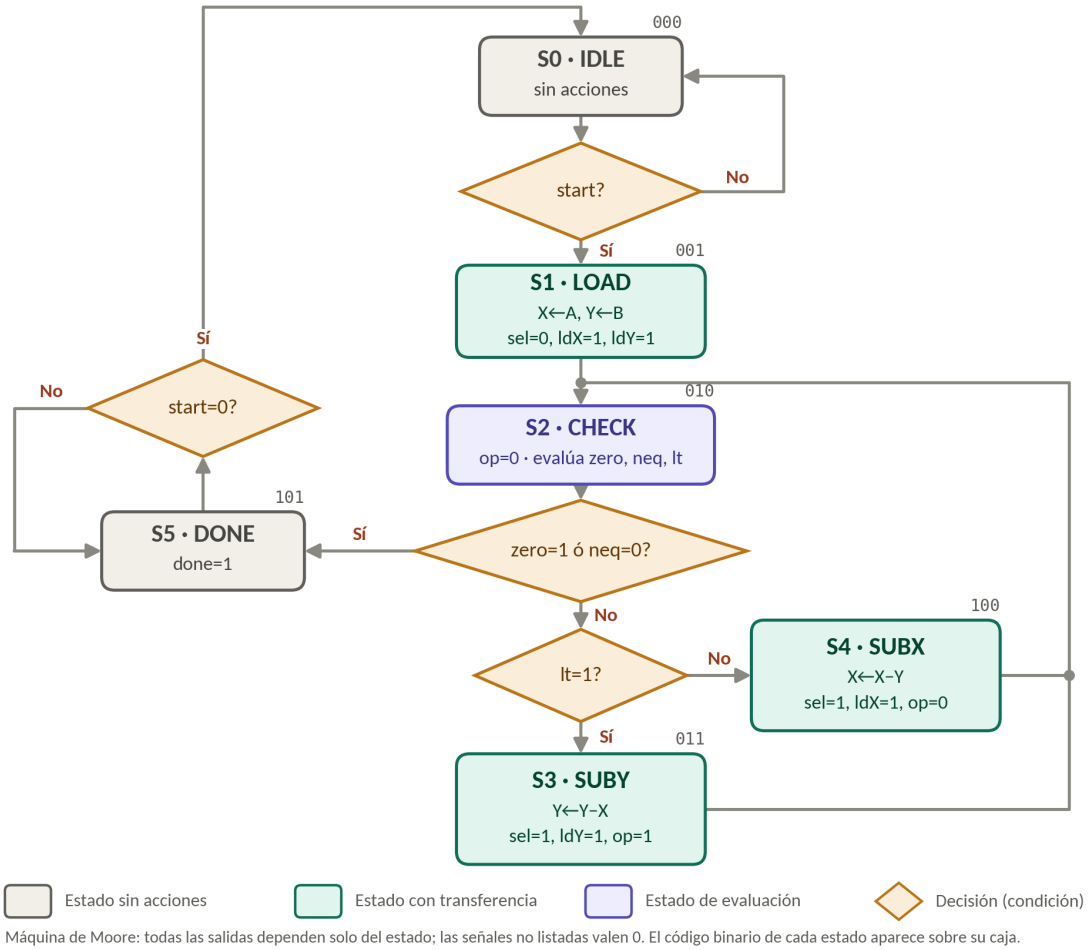


Figura 1: Maquina algorítmica de estados propuesta.

4. Camino de datos

Para la construcción del camino de datos (*datapath*), se estableció que los datos de entrada X y Y tendrían un ancho de bus de 8 bits sin signo (*unsigned*), dado que el algoritmo del Máximo Común Divisor opera sobre

números enteros positivos. Tras iterar en el proceso de diseño y optimizar los bloques funcionales, se obtuvo la arquitectura mostrada en la Figura 2.

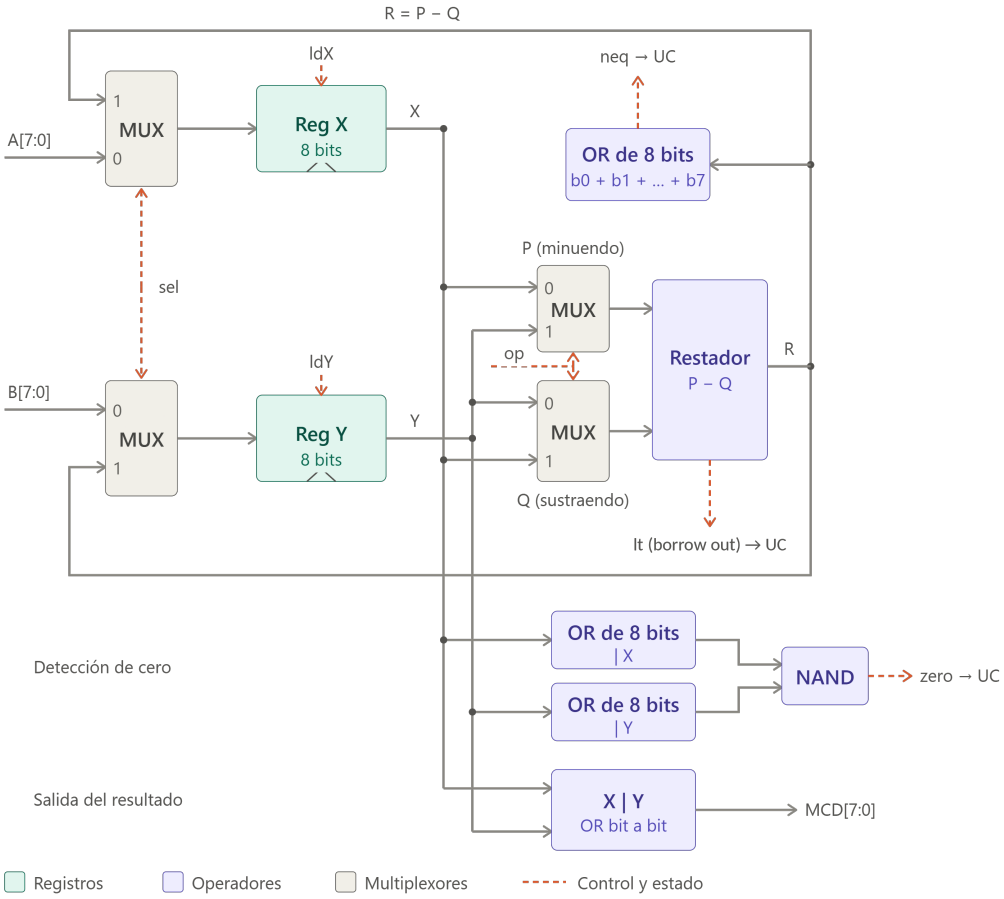


Figura 2: Camino de datos propuesto.

Como se observa en la Figura 2, se optimizó la estructura unificando las operaciones de resta en un único bloque restador ($R = P - Q$). La selección del minuendo (P) y del sustraendo (Q) se realiza mediante dos multiplexores de entrada multiplexados por la señal de control `op`:

- Con `op = 0`: $P = X$ y $Q = Y$, realizando la operación $R = X - Y$.
- Con `op = 1`: $P = Y$ y $Q = X$, realizando la operación $R = Y - X$.

Esta simplificación reduce significativamente los recursos de hardware necesarios en comparación con el uso de dos restadores independientes. Adicionalmente, el restador se aprovecha para generar las señales de estado hacia la unidad de control (UC):

- **Señal `lt` (*less than*):** Corresponde directamente al acarreo de salida o préstamo (*borrow out*) del restador, indicando si el resultado de la comparación/resta es negativo ($X < Y$).
- **Señal `neq` (*not equal*):** Se obtiene aplicando una operación de reducción OR sobre los 8 bits del resultado R ($b_0 + b_1 + \cdots + b_7$). Si $R = 0$, la señal toma el valor de 0 (indicando $X = Y$); de lo contrario, toma el valor 1, indicando a la UC que debe continuar las iteraciones del algoritmo.

Para gestionar los casos límite en los que una de las entradas es igual a cero (por ejemplo, $\text{MCD}(0, 5) = 5$), se incorporó un módulo de **detección de cero**. Este bloque calcula la reducción OR de cada variable ($|X$ y $|Y$) y combina ambos resultados mediante una compuerta NAND, generando la señal `zero`. Esta detección evita que el sistema entre en un bucle infinito al realizar restas sucesivas por cero.

Por último, la **salida del resultado** (`MCD[7:0]`) se genera mediante una operación de OR bit a bit entre X e Y ($X \mid Y$). Esta estructura garantiza la entrega del resultado correcto tanto al finalizar el ciclo cuando $X = Y$, como cuando una de las entradas iniciales es 0.

Las señales de control `ldX` y `ldY` habilitan la carga de nuevos datos en los registros **Reg X** y **Reg Y**, mientras que la señal `sel` conmuta los multiplexores de entrada entre las variables externas ($A[7:0], B[7:0]$ para `sel = 0`) y el resultado de la resta realimentado (R para `sel = 1`).

4.1. Restador

Para la implementación del restador completo de 8 bits, se partió del diseño de un módulo básico de resta de 1 bit, cuyo comportamiento lógico se describe mediante la Tabla 1.

a	b	b_{in}	d	b_{out}
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

Cuadro 1: Tabla de verdad de un restador completo de 1 bit.

La función del bloque responde a la operación $d = a - b - b_{in}$, donde b_{in} (*borrow in*) representa el préstamo pendiente recibido del bit menos significativo (LSB), y b_{out} (*borrow out*) indica si la posición actual requiere generar un préstamo hacia el bit más significativo (MSB).

A modo de analogía con la aritmética decimal, al restar $42 - 17$, en la posición de las unidades $(2 - 7)$ se requiere pedir prestada una base al dígito superior, resultando en $(2+10)-7 = 5$. En la posición de las decenas, el dígito minuendo disminuye por el préstamo previo, resultando en $(4 - 1) - 1 = 2$, para un resultado final de 25. En el caso binario ocurre el mismo principio con base 2: b_{in} descuenta la unidad cedida en la etapa previa y b_{out} señala la necesidad de solicitar un préstamo a la etapa siguiente.

A partir de la Tabla 1, mediante la simplificación con mapas de Karnaugh, se obtienen las expresiones booleanas para la diferencia (d) y el préstamo de salida (b_{out}):

$$d = a \oplus b \oplus b_{in}$$

$$b_{out} = \overline{a}b + b_{in}(\overline{a} + b)$$

A nivel de descripción de hardware, este módulo de 1 bit se modeló en Verilog como se muestra a continuación:

```
module Restador_1bit (
    input  a,      // Minuendo
    input  b,      // Sustraendo
    input  bin,    // Borrow de entrada
    output d,      // Diferencia
    output bout    // Borrow de salida
);
    assign d    = a ^ b ^ bin;
    assign bout = (~a & b) | ((~a | b) & bin);
endmodule
```

4.2. Restador de 8 bits

Haciendo uso del módulo de 1 bit como bloque constructivo fundamental, es posible ascender en el nivel de abstracción mediante una arquitectura jerárquica para conformar un restador de 8 bits por propagación de préstamo (*ripple-borrow subtractor*), tal como se ilustra en la Figura 3.

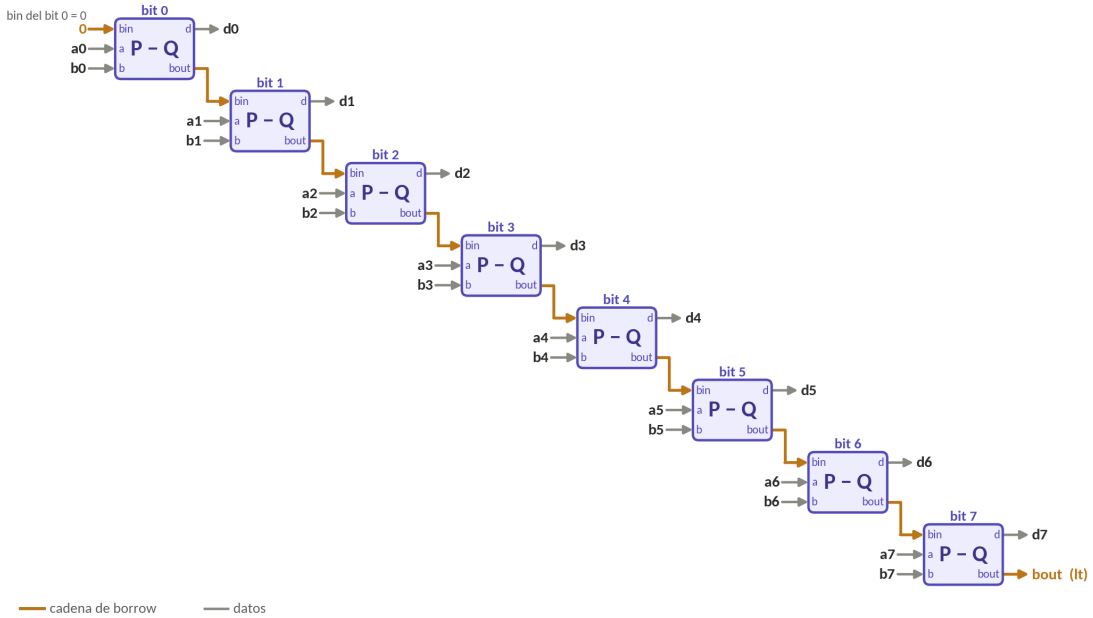


Figura 3: Restador de 8 bits conformado por la cascada de restadores de 1 bit.

En este esquema, los bits de entrada de $a[7:0]$ y $b[7:0]$ se procesan posición a posición de menor a mayor significancia. El préstamo generado por el bit i (`borrow[i+1]`) se conecta directamente como entrada del préstamo (`bin`) del bit $i+1$. El bit menos significativo no recibe préstamo inicial (`borrow[0] = 0`), mientras que el préstamo resultante del bit más significativo (`borrow[8]`) determina el *borrow out* general (`bout`), señal que indica si $a < b$ (equivalente a la señal de estado `lt` hacia la unidad de control).

La descripción estructural correspondiente en Verilog se presenta a continuación:

```
// Restador de 8 bits por propagacion de prestamo (ripple-borrow): D = A - B
// bout = 1 cuando A < B (hubo prestamo en el bit mas significativo)
module Restador_8bit (
    input  [7:0] a,      // Minuendo
    input  [7:0] b,      // Sustraendo
    output [7:0] d,      // Diferencia
    output       bout    // Borrow de salida (senal lt)
);
    wire [8:0] borrow;   // Bus intermedio de prestamos entre etapas

    assign borrow[0] = 1'b0; // El bit 0 no recibe prestamo de entrada

    // Instanciacion en cascada de los restadores de 1 bit
    Restador_1bit rb0 (.a(a[0]), .b(b[0]), .bin(borrow[0]), .d(d[0]), .bout(borrow[1]));
    Restador_1bit rb1 (.a(a[1]), .b(b[1]), .bin(borrow[1]), .d(d[1]), .bout(borrow[2]));
    Restador_1bit rb2 (.a(a[2]), .b(b[2]), .bin(borrow[2]), .d(d[2]), .bout(borrow[3]));
    Restador_1bit rb3 (.a(a[3]), .b(b[3]), .bin(borrow[3]), .d(d[3]), .bout(borrow[4]));
    Restador_1bit rb4 (.a(a[4]), .b(b[4]), .bin(borrow[4]), .d(d[4]), .bout(borrow[5]));
    Restador_1bit rb5 (.a(a[5]), .b(b[5]), .bin(borrow[5]), .d(d[5]), .bout(borrow[6]));
    Restador_1bit rb6 (.a(a[6]), .b(b[6]), .bin(borrow[6]), .d(d[6]), .bout(borrow[7]));
    Restador_1bit rb7 (.a(a[7]), .b(b[7]), .bin(borrow[7]), .d(d[7]), .bout(borrow[8]));

    assign bout = borrow[8];
endmodule
```

4.3. Registro de 8 bits

Para el almacenamiento temporal de los operandos X e Y, se diseñó un registro de carga paralela de 8 bits con señal de habilitación (*load enable*) y reinicio síncrono activo en alto (*synchronous reset*).

La arquitectura conceptual del bloque se ilustra en la Figura 4, mientras que su implementación a un menor nivel de abstracción, compuesta por multiplexores de entrada y flip-flops tipo D conectados en paralelo, se

presenta en la Figura 5.

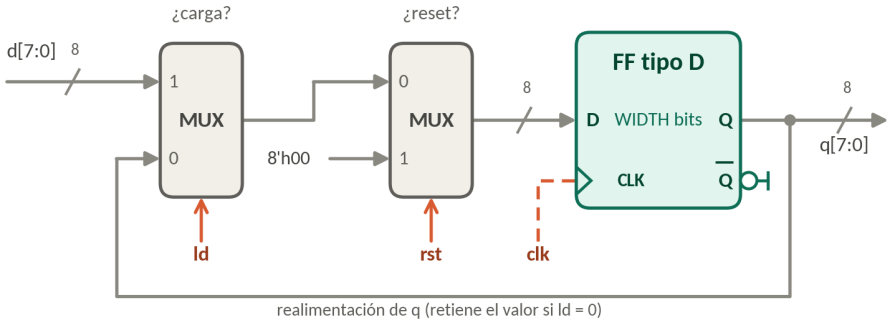


Figura 4: Bloque funcional del registro de 8 bits con carga habilitada.

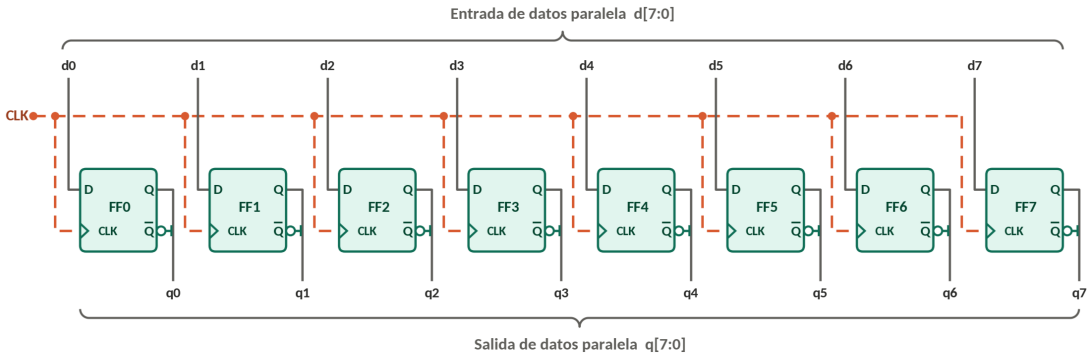


Figura 5: Estructura interna del registro en paralelo a nivel de flip-flops tipo D.

En cada flanco de subida del reloj (`posedge clk`), el registro evalúa prioritariamente la señal de reinicio `rst`. Si esta se encuentra activa (`rst = 1`), la salida `q` se reestablece a cero. De lo contrario, si la señal de habilitación está activa (`ld = 1`), se actualiza la salida con el dato de entrada `d`. Cuando `ld = 0`, el registro retroalimenta su propia salida manteniendo el valor almacenado sin cambios.

A continuación se presenta la descripción de hardware correspondiente en Verilog, parametrizada para un ancho de palabra por defecto de 8 bits:

```
module Reg_8bit #(
    parameter WIDTH = 8
)(
    input  wire           clk,
    input  wire           rst,   // Reset sincrono (activo en alto)
    input  wire           ld,    // Habilita la carga
    input  wire [WIDTH-1:0] d,   // Entrada de datos
    output reg [WIDTH-1:0] q     // Salida de datos
);

    always @(posedge clk) begin
        if (rst)
            q <= {WIDTH{1'b0}};
        else if (ld)
            q <= d;
        // Si ld = 0, q conserva su valor previo
    end

endmodule
```

4.4. Reducción OR de 8 bits y Multiplexor

Para implementar la detección de cero o desigualdad a partir de buses de datos de 8 bits, se diseñó un módulo de **reducción OR** mediante una estructura de árbol binario balanceado de tres niveles. En el primer nivel, se evalúan en paralelo pares contiguos de bits ($a[0]$–$a[1]$, $a[2]$–$a[3]$, $a[4]$–$a[5]$, $a[6]$–$a[7]$). Los cuatro resultados intermedios se combinan en un segundo nivel de compuertas OR, entregando dos señales que finalmente se resumen en una compuerta OR de salida. Esta topología en árbol minimiza el retardo de propagación total en comparación con una cadena secuencial.

A continuación, se presenta la descripción estructural de la compuerta OR de 8 bits en Verilog:

```
module Or_8bit (
    input  [7:0] a,  // Bus de entrada de 8 bits
    output       y   // Salida escalar (1 si al menos un bit es 1)
);
    wire n0, n1, n2, n3; // Nivel 1 (4 compuertas OR)
    wire m0, m1;         // Nivel 2 (2 compuertas OR)

    assign n0 = a[0] | a[1];
    assign n1 = a[2] | a[3];
    assign n2 = a[4] | a[5];
    assign n3 = a[6] | a[7];

    assign m0 = n0 | n1;
    assign m1 = n2 | n3;

    assign y = m0 | m1;  // Nivel 3 (Salida final)
endmodule
```

Adicionalmente, para la conmutación de datos en el sistema (tanto en las entradas del restador como a la entrada de los registros), se modeló un **multiplexor de 2 a 1 parametrizado** mediante el operador condicional ternario. Cuando la señal de selección toma el valor `sel = 0`, la salida entrega el canal `d0`; en caso contrario (`sel = 1`), se conmuta hacia la entrada `d1`.

Su implementación parametrizada en Verilog se describe a continuación:

```
module mux2 #(
    parameter WIDTH = 8  // Ancho de palabra configurable
)(
    input wire [WIDTH-1:0] d0, // Entrada seleccionada cuando sel = 0
    input wire [WIDTH-1:0] d1, // Entrada seleccionada cuando sel = 1
    input wire             sel, // Senal de seleccion
    output wire [WIDTH-1:0] y  // Salida de datos multiplexada
);

    assign y = sel ? d1 : d0;

endmodule
```

4.5. Integración estructural del Camino de Datos

Integrando todos los bloques funcionales previamente descritos (multiplexores, registros, restador y lógica de reducción OR), se interconectó el camino de datos topológico (*datapath*) del sistema. Este módulo administra el flujo de datos principal y se comunica con la Unidad de Control mediante dos conjuntos de señales:

- **Señales de control (entrantes de la UC):** `sel` (intercambio entre entradas externas y realimentación), `ldX`/`ldY` (habilitación de carga en registros) y `op` (conmutación de minuendo y sustraendo en el restador).
- **Señales de estado (salientes hacia la UC):** `neq` (indica si $X \neq Y$), `lt` (indica préstamo/resultado negativo $P < Q$) y `zero` (indica si $X = 0$ o $Y = 0$ para prevenir bucles infinitos).

La descripción jerárquica completa en Verilog se presenta a continuación:

```
module datapath (
    input wire      clk,
    input wire      rst,
    input wire [7:0] A,
    input wire [7:0] B,
```

```
6
7       // Senales de control procedentes de la UC
8       input wire      sel,  // 0: carga A/B, 1: carga resultado R
9       input wire      ldX,  // Habilita registro X
10      input wire      ldY,  // Habilita registro Y
11      input wire      op,   // 0: P-Q = X-Y, 1: P-Q = Y-X
12
13      // Senales de estado dirigidas a la UC
14      output wire     neq,  // 1 si X != Y
15      output wire     lt,   // 1 si P < Q (borrow out del restador)
16      output wire     zero, // 1 si X == 0 o Y == 0
17
18      // Salida final del resultado
19      output wire [7:0] MCD
20  );
21
22      wire [7:0] X, Y;      // Salidas de los registros
23      wire [7:0] x_in, y_in; // Entradas a los registros
24      wire [7:0] P, Q, R;   // Operandos y resultado del restador
25      wire      or_x, or_y; // Salidas de reduccion OR (deteccion de cero)
26
27      // ---------------- Multiplexores de entrada a registros ----------------
28      mux2 #(.WIDTH(8)) mux_x (.d0(A), .d1(R), .sel(sel), .y(x_in));
29      mux2 #(.WIDTH(8)) mux_y (.d0(B), .d1(R), .sel(sel), .y(y_in));
30
31      // ---------------- Registros de almacenamiento ----------------
32      Reg_8bit #(.WIDTH(8)) regX (.clk(clk), .rst(rst), .ld(ldX), .d(x_in), .q(X));
33      Reg_8bit #(.WIDTH(8)) regY (.clk(clk), .rst(rst), .ld(ldY), .d(y_in), .q(Y));
34
35      // ---------------- Multiplexores de operandos para el restador ----------------
36      mux2 #(.WIDTH(8)) mux_p (.d0(X), .d1(Y), .sel(op), .y(P)); // Minuendo
37      mux2 #(.WIDTH(8)) mux_q (.d0(Y), .d1(X), .sel(op), .y(Q)); // Sustraendo
38
39      // ---------------- Submdulo restador: R = P - Q ----------------
40      Restador_8bit sub (.a(P), .b(Q), .d(R), .bout(lt));
41
42      // ---------------- Generacin de neq: Reduccin OR de R ----------------
43      Or_8bit or_r (.a(R), .y(neq));
44
45      // ---------------- Deteccin de condicion cero: NAND(|X, |Y) ----------------
46      Or_8bit or_xx (.a(X), .y(or_x));
47      Or_8bit or_yy (.a(Y), .y(or_y));
48      assign zero = ~(or_x & or_y);
49
50      // ---------------- Bus de salida: Operacin OR bit a bit (X | Y) ----------------
51      assign MCD = X | Y;
52
53  endmodule
```

5. Máquina de estados de la unidad de control

A partir de la ASM definida previamente (Figura 1), se construyó el diagrama de transición de la Máquina de Estados Finitos (FSM, *Finite State Machine*) de tipo Moore que rige la unidad de control, ilustrada en la Figura 6. Esta máquina secuencia las operaciones del camino de datos a lo largo de 6 estados principales (**S0: IDLE**, **S1: LOAD**, **S2: CHECK**, **S3: SUBY**, **S4: SUBX** y **S5: DONE**) en función de las señales externas y de las condiciones evaluadas en el sistema.

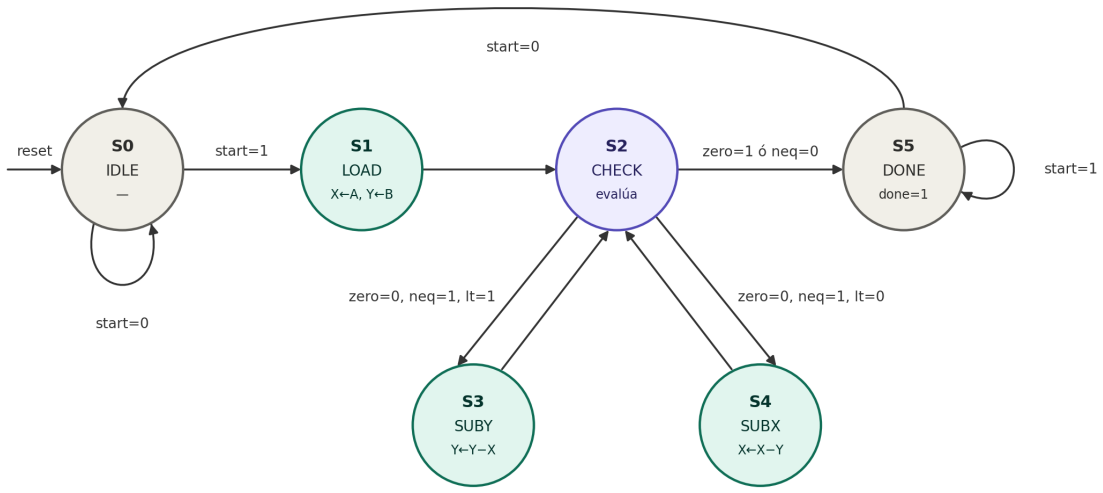


Figura 6: Máquina de estados finita (FSM) de la unidad de control del sistema.

Las señales involucradas en la interacción entre la unidad de control, el usuario y el camino de datos se resumen a continuación categorizadas por su función:

Señales de entrada de control (externas)

- `reset`**:** Señal de reinicio global asíncrono o síncrono que ubica de manera forzada a la FSM en el estado de reposo inicial (**S0** · **IDLE**).
- `start`**:** Señal de inicio de operación proporcionada por el usuario. Un pulso en alto (`start = 1`) inicia la transición desde **S0** hacia la fase de carga (**S1**). Mientras permanezca activa en **S5**, el sistema se mantiene en el estado final; únicamente cuando vuelve a cero (`start = 0`), la FSM retorna a **S0** · **IDLE**.

Señales de estado o banderas (procedentes del datapath)

- `zero`**:** Bandera de detección de valor nulo ($X = 0$ o $Y = 0$). Si `zero = 1`, indica que al menos uno de los operandos es cero, lo que conmuta el flujo en **S2** · **CHECK** directamente hacia el estado final **S5** · **DONE** para evitar restas indefinidas.
- `neq` **(*not equal*):** Indicación de desigualdad entre operandos ($X \neq Y$). Si `neq = 0` (es decir, $X = Y$), se ha alcanzado la condición de paro del algoritmo de Euclides, transitando hacia **S5** · **DONE**.
- `lt` **(*less than*):** Señal de comparación de orden procedente del *borrow out* del restador ($X < Y$). En el estado **S2** · **CHECK**, si `lt = 1` se transita a **S3** · **SUBY** ($Y \leftarrow Y - X$); si `lt = 0` ($X \geq Y$), se transita a **S4** · **SUBX** ($X \leftarrow X - Y$).

Señales de salida de control (hacia el datapath y el usuario)

- `sel`**:** Controla los multiplexores de entrada a los registros. Toma el valor `0` durante **S1** · **LOAD** para cargar los datos primarios A y B, y `1` en los estados **S3** y **S4** para realimentar el resultado de la resta R.
- `ldX` / `ldY`**:** Habilitadores de escritura de los registros **Reg X** y **Reg Y**. Se activan según corresponda para actualizar X (`ldX = 1`) o Y (`ldY = 1`) durante la carga inicial o tras efectuar una resta.
- `op`**:** Selecciona la orientación de los operandos en el restador unificado. Fija $P = X, Q = Y$ $(X - Y)$ cuando `op = 0`, y $P = Y, Q = X$ $(Y - X)$ cuando `op = 1`.

- `done`: Bandera externa de finalización de proceso. Se activa únicamente en el estado **S5 · DONE** (`done = 1`) para notificar que el resultado válido del MCD está disponible en la salida.

A partir del diagrama de la máquina de estados finitos (FSM), se construyó la tabla de transiciones (Tabla 2), la cual define el comportamiento del estado siguiente (Q_2^+, Q_1^+, Q_0^+) en función del estado actual y de las señales de entrada/estado (*start*, *zero*, *neq*, *lt*).

Cuadro 2: Tabla de transiciones de la FSM (estado siguiente).

Estado actual	Q_2	Q_1	Q_0	*start*	*zero*	*neq*	*lt*	**Siguiente estado**	Q_2^+	Q_1^+	Q_0^+
S0 IDLE	0	0	0	0	x	x	x	S0 IDLE	0	0	0
S0 IDLE	0	0	0	1	x	x	x	S1 LOAD	0	0	1
S1 LOAD	0	0	1	x	x	x	x	S2 CHECK	0	1	0
S2 CHECK	0	1	0	x	1	x	x	S5 DONE	1	0	1
S2 CHECK	0	1	0	x	0	0	x	S5 DONE	1	0	1
S2 CHECK	0	1	0	x	0	1	1	S3 SUBY	0	1	1
S2 CHECK	0	1	0	x	0	1	0	S4 SUBX	1	0	0
S3 SUBY	0	1	1	x	x	x	x	S2 CHECK	0	1	0
S4 SUBX	1	0	0	x	x	x	x	S2 CHECK	0	1	0
S5 DONE	1	0	1	0	x	x	x	S0 IDLE	0	0	0
S5 DONE	1	0	1	1	x	x	x	S5 DONE	1	0	1

Asimismo, debido a que se trata de una máquina de estado tipo Moore, las señales de control y de salida hacia el camino de datos y el usuario dependen únicamente de la codificación del estado actual (Q_2, Q_1, Q_0), como se detalla en la Tabla 3.

Cuadro 3: Tabla de salidas de la FSM según la codificación de estado.

Estado	Q_2	Q_1	Q_0	**Código**	*sel*	*op*	*ldX*	*ldY*	*done*
S0 IDLE	0	0	0	000	0	0	0	0	0
S1 LOAD	0	0	1	001	0	0	1	1	0
S2 CHECK	0	1	0	010	0	0	0	0	0
S3 SUBY	0	1	1	011	1	1	0	1	0
S4 SUBX	1	0	0	100	1	0	1	0	0
S5 DONE	1	0	1	101	0	0	0	0	1

A partir de la interpretación directa de los estados y la lógica combinacional de la FSM, se dedujeron las ecuaciones booleanas para los estados futuros (en función del tiempo $t+1$) y para los canales de control de salida:

Ecuaciones de transición de estado

$$
\begin{aligned}
\text{IDLE}_{(t+1)} &= \text{reset} + (\text{IDLE} \cdot \overline{\text{start}}) + (\text{DONE} \cdot \overline{\text{start}}) \\
\text{LOAD}_{(t+1)} &= \text{IDLE} \cdot \text{start} \\
\text{CHECK}_{(t+1)} &= \text{LOAD} + \text{SUBY} + \text{SUBX} \\
\text{SUBY}_{(t+1)} &= \text{CHECK} \cdot \overline{\text{zero}} \cdot \text{neq} \cdot \text{lt} \\
\text{SUBX}_{(t+1)} &= \text{CHECK} \cdot \overline{\text{zero}} \cdot \text{neq} \cdot \overline{\text{lt}} \\
\text{DONE}_{(t+1)} &= \text{CHECK} \cdot (\text{zero} + \overline{\text{neq}}) + (\text{DONE} \cdot \text{start})
\end{aligned}
$$

Ecuaciones de las señales de salida

$$
\begin{aligned}
sel &= \text{SUBY} + \text{SUBX} \\
op &= \text{SUBY} \\
ldX &= \text{LOAD} + \text{SUBX} \\
ldY &= \text{LOAD} + \text{SUBY} \\
done &= \text{DONE}
\end{aligned}
$$

A nivel de implementación lógica, la Unidad de Control se puede estructurar en un menor nivel de abstracción mediante tres flip-flops de tipo D para almacenar la codificación del estado actual (Q_2, Q_1, Q_0), acompañados por bloques de lógica combinacional para la transición de estado y la generación de salidas, como se ilustra en la Figura 7.

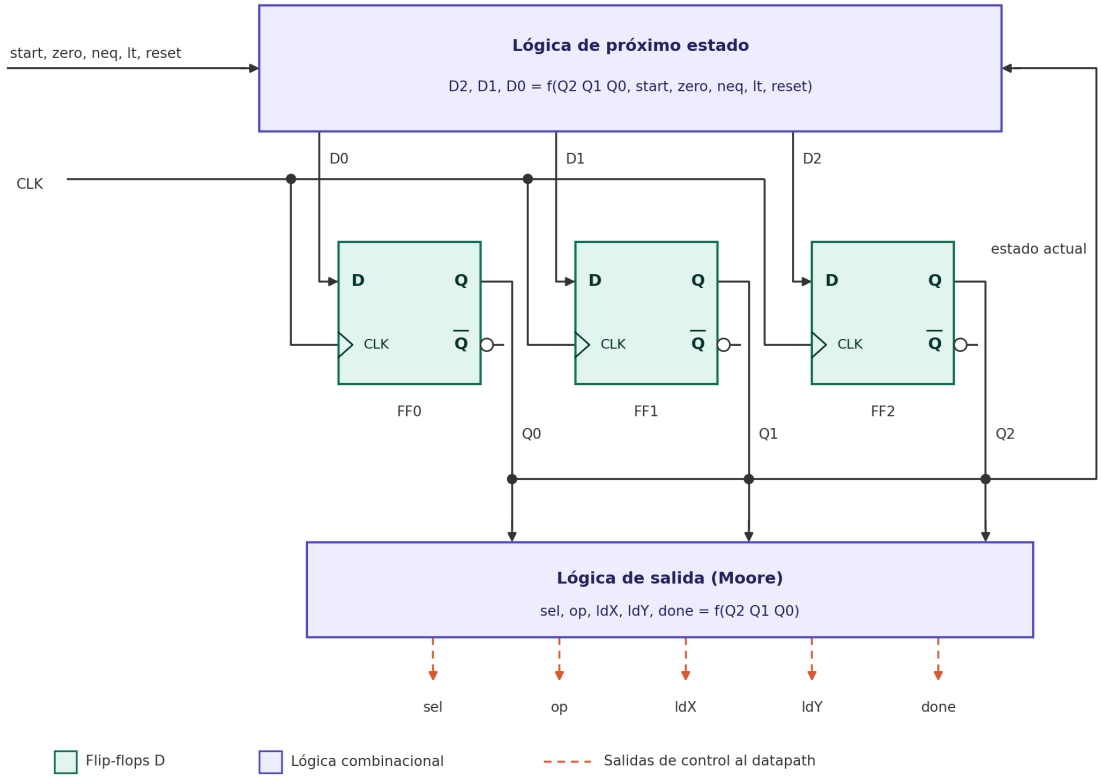


Figura 7: Esquemático digital de la máquina de estados a nivel de registro de estado (3 Flip-Flops D) y bloques combinacionales de próximo estado y salida.

Finalmente, la descripción en Verilog de la Unidad de Control se modeló siguiendo la metodología estándar de tres bloques (*3-always block design*): el registro de estado con reseteo síncrono activo en alto, la lógica combinacional para el cálculo del estado siguiente y el decodificador de salidas de tipo Moore.

La descripción HDL correspondiente se presenta a continuación:

```
1  module fsm (
2      input wire clk,
3      input wire rst,    // Reset sincrono, activo en alto
4      input wire start,  // Senal de inicio de operacion
5
6      // Banderas de estado procedentes del datapath
```

```
    input wire zero,
    input wire neq,
    input wire lt,

    // Senales de control hacia el datapath
    output reg sel,
    output reg ldX,
    output reg ldY,
    output reg op,

    // Indicador externo de finalizacion
    output reg done
);

    // Codificacion de estados (3 bits)
    localparam [2:0] S0_IDLE = 3'd0,
                     S1_LOAD = 3'd1,
                     S2_CHECK = 3'd2,
                     S3_SUBY = 3'd3,
                     S4_SUBX = 3'd4,
                     S5_DONE = 3'd5;

    reg [2:0] state, next_state;

    // ---------------- Bloque 1: Registro de estado (Secuencial) ----------------
    always @(posedge clk) begin
        if (rst)
            state <= S0_IDLE;
        else
            state <= next_state;
    end

    // ---------------- Bloque 2: Logica de siguiente estado (Combinacional) ----------------
    always @(*) begin
        next_state = state; // Asignacion por defecto para evitar latches
        case (state)
            S0_IDLE: if (start) next_state = S1_LOAD;

            S1_LOAD: next_state = S2_CHECK;

            S2_CHECK: begin
                if (zero || !neq)
                    next_state = S5_DONE;
                else if (lt)
                    next_state = S3_SUBY; // X < Y
                else
                    next_state = S4_SUBX; // X >= Y
            end

            S3_SUBY: next_state = S2_CHECK;
            S4_SUBX: next_state = S2_CHECK;

            S5_DONE: if (!start) next_state = S0_IDLE;

            default: next_state = S0_IDLE;
        endcase
    end

    // ---------------- Bloque 3: Logica de salidas tipo Moore (Combinacional) ----------------
    always @(*) begin
        // Valores por defecto (previenen la inferencia de latches)
        sel  = 1'b0;
        ldX  = 1'b0;
        ldY  = 1'b0;
        op   = 1'b0;
        done = 1'b0;

        case (state)
            S1_LOAD: begin
```

```
                sel = 1'b0; ldX = 1'b1; ldY = 1'b1;  // Carga inicial: X <- A, Y <- B
            end

            S3_SUBY: begin
                sel = 1'b1; ldY = 1'b1; op = 1'b1;   // Actualizacion: Y <- Y - X
            end

            S4_SUBX: begin
                sel = 1'b1; ldX = 1'b1; op = 1'b0;   // Actualizacion: X <- X - Y
            end

            S5_DONE: begin
                done = 1'b1;                         // Notificacion de fin de ejecucion
            end

            default: ; // En S0 y S2 todas las salidas permanecen inactivas en 0
        endcase
    end

endmodule
```

6. Resultados

6.1. Integración del camino de datos y la unidad de control

Para consolidar la arquitectura completa del procesador del Máximo Común Divisor (MCD), se diseñó el módulo de nivel superior (*top-level*) denominado `mcd_top`. Este bloque interconecta estructuralmente la Unidad de Control (`fsm`) y el Camino de Datos (`datapath`) mediante los buses intermedios de control y estado:

- **Lógica de control (`fsm` → `datapath`):** Genera las señales `sel`, `ldX`, `ldY` y `op` para gobernar el flujo de datos y la ejecución de restas sucesivas.
- **Lógica de estado (`datapath` → `fsm`):** Realimenta las banderas `zero`, `neq` y `lt` para la toma de decisiones en cada ciclo de reloj.

Esta integración jerárquica modela con precisión el comportamiento completo expresado en la ASM inicial (Figura 1) y en el diagrama FSM (Figura 6).

A continuación, se presenta la descripción estructural en Verilog del módulo top-level:

```
module mcd_top (
    input wire      clk,   // Senal de reloj global
    input wire      rst,   // Reset sincrono
    input wire      start, // Inicio de calculo
    input wire [7:0] A,    // Operando A de entrada
    input wire [7:0] B,    // Operando B de entrada
    output wire [7:0] MCD, // Resultado del Maximo Comun Divisor
    output wire     done   // Indicador de finalizacion de proceso
);

    // Interconexiones de control: FSM -> Datapath
    wire sel, ldX, ldY, op;

    // Interconexiones de estado: Datapath -> FSM
    wire zero, neq, lt;

    // Instanciacion de la Unidad de Control (FSM)
    fsm ctrl (
        .clk (clk),
        .rst (rst),
        .start(start),
        .zero (zero),
        .neq  (neq),
        .lt   (lt),
        .sel  (sel),
        .ldX  (ldX),
```

```
        .ldY (ldY),
        .op  (op),
        .done (done)
    );

    // Instanciacion del Camino de Datos (Datapath)
    datapath dp (
        .clk (clk),
        .rst (rst),
        .A   (A),
        .B   (B),
        .sel (sel),
        .ldX (ldX),
        .ldY (ldY),
        .op  (op),
        .neq (neq),
        .lt  (lt),
        .zero(zero),
        .MCD (MCD)
    );

endmodule
```

6.2. Simulación

Con el fin de verificar el funcionamiento del sistema antes de realizar su implementación física, se desarrolló un banco de pruebas automatizado (*testbench*) empleando Icarus Verilog. La simulación permitió evaluar tanto los módulos individuales como la integración entre la unidad de control y el camino de datos.

Inicialmente se verificó el funcionamiento del núcleo encargado del cálculo del Máximo Común Divisor, aplicando diferentes combinaciones de operandos de 8 bits. Para cada vector de prueba, el resultado obtenido por el diseño fue comparado automáticamente con el valor esperado, permitiendo detectar errores sin necesidad de realizar una inspección manual de cada caso.

Entre los casos utilizados durante la verificación se incluyeron valores con divisores comunes, números coprimos, operandos iguales y valores nulos. Algunos de los casos representativos se muestran en la Tabla 4.

Cuadro 4: Casos representativos utilizados durante la simulación funcional.

A	**B**	**MCD esperado**	**Resultado**
48	18	6	Correcto
12	8	4	Correcto
15	10	5	Correcto
21	14	7	Correcto
25	12	1	Correcto
36	36	36	Correcto
0	5	5	Correcto

En particular, para el caso $A = 48$ y $B = 18$, la FSM carga inicialmente ambos operandos en los registros X y Y y posteriormente ejecuta las restas sucesivas definidas por el algoritmo de Euclides. Después de las iteraciones correspondientes, los operandos convergen al valor 6, momento en el cual la señal `neq` se desactiva y la unidad de control transita al estado **DONE**, activando la señal `done`.

Además de la verificación del núcleo MCD, se realizó una simulación del sistema integrado incluyendo la interfaz de entrada. En esta prueba se emularon los pulsos provenientes del panel táctil y del pulsador físico. El sistema permitió introducir $A = 48$ y $B = 18$, generar automáticamente el pulso de `start` y obtener como resultado final $MCD = 6$, verificándose también la transición de los estados de interfaz desde la captura del operando A, la captura del operando B, la ejecución y finalmente la visualización del resultado.

Las formas de onda obtenidas permitieron comprobar la correcta sincronización entre las señales de control `sel`, `ldX`, `ldY` y `op`, las señales de estado `zero`, `neq` y `lt`, y las transiciones de la FSM.

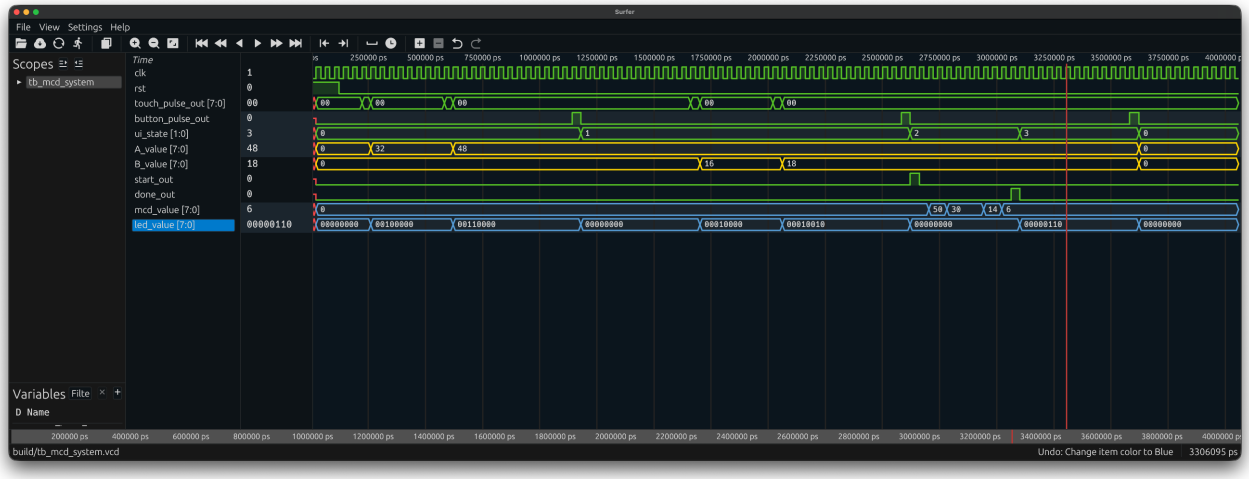


Figura 8: Simulación funcional del sistema completo para el cálculo de MCD$(48, 18) = 6$. Se observa el ingreso secuencial de los operandos, la generación de la señal de inicio, la ejecución del algoritmo y la presentación binaria del resultado en la salida de LED.

6.3. Circuito sintetizado

Una vez validado funcionalmente el diseño, se realizó la síntesis lógica para la FPGA Lattice ECP5 LFE5U-45F presente en la tarjeta Colorlight i9. Para este proceso se utilizó Yosys como herramienta de síntesis, mientras que la etapa de *place and route* fue realizada mediante `nextpnr-ecp5`.

La Figura 9 presenta la vista RTL jerárquica del núcleo `mcd_top`. En ella se observa claramente la separación planteada desde la metodología de diseño: la unidad de control, implementada por el módulo `fsm`, y el camino de datos, implementado mediante el módulo `datapath`. Entre ambos bloques se encuentran las señales de control `sel`, `ldX`, `ldY` y `op`, y las señales de estado `zero`, `neq` y `lt`.

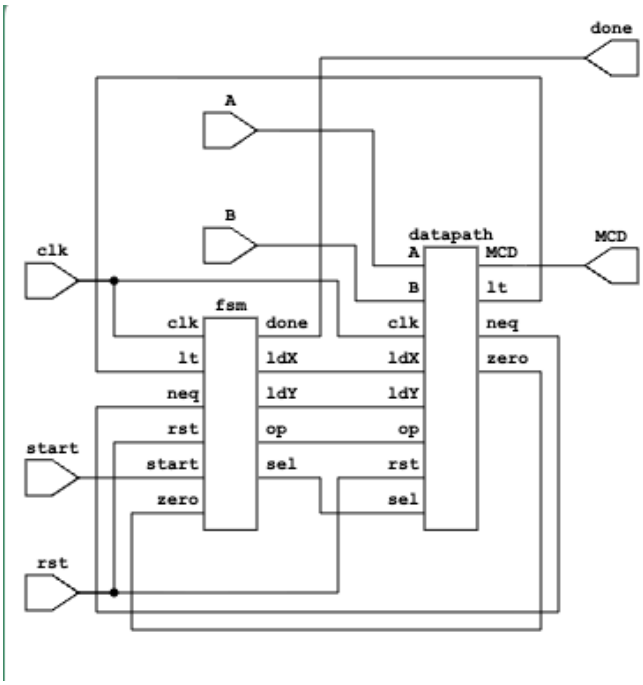


Figura 9: Vista RTL jerárquica del núcleo `mcd_top`. Se observa la interconexión entre la unidad de control (`fsm`) y el camino de datos (`datapath`), junto con las señales de control y estado.

Para la implementación física se añadió una capa superior denominada `top_colorlight`, encargada de adaptar el núcleo MCD a los periféricos reales utilizados en la tarjeta FPGA. Esta arquitectura incorpora módulos de sincronización para las entradas asíncronas, detección de flancos para el panel táctil, eliminación de rebotes del pulsador y una unidad de control de interfaz encargada de almacenar los operandos introducidos por el usuario y seleccionar la información presentada en el arreglo de LED.

La vista jerárquica del sistema completo se presenta en la Figura 10. Esta permite observar que el núcleo del MCD se mantiene independiente de la lógica específica de la plataforma, facilitando su reutilización en otras tarjetas FPGA.

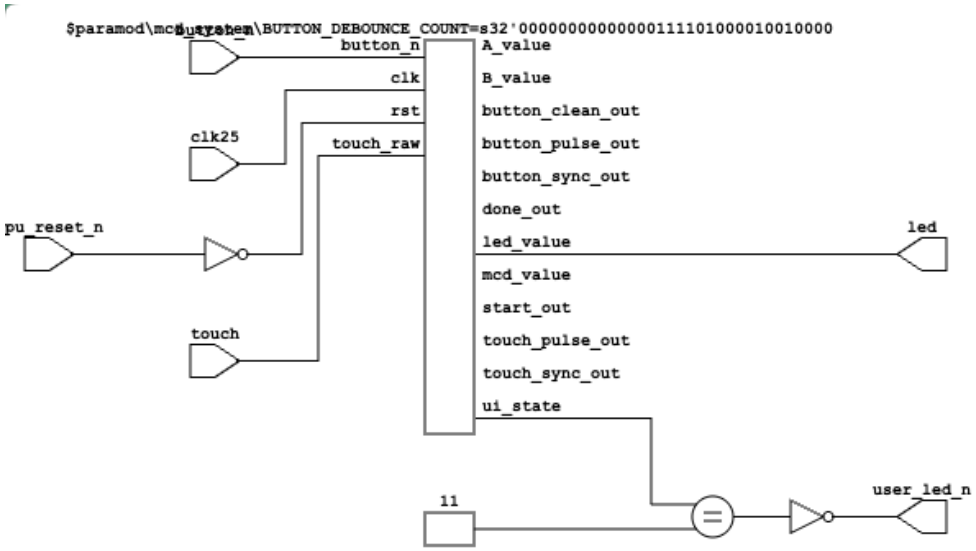


Figura 10: Vista RTL jerárquica del sistema completo implementado sobre la Colorlight i9. Se observan los módulos principales de interfaz, control y procesamiento del MCD.

La Figura 10 muestra la organización jerárquica del sistema a nivel RTL, conservando la separación funcional entre los módulos de entrada, control de usuario y núcleo de cálculo.

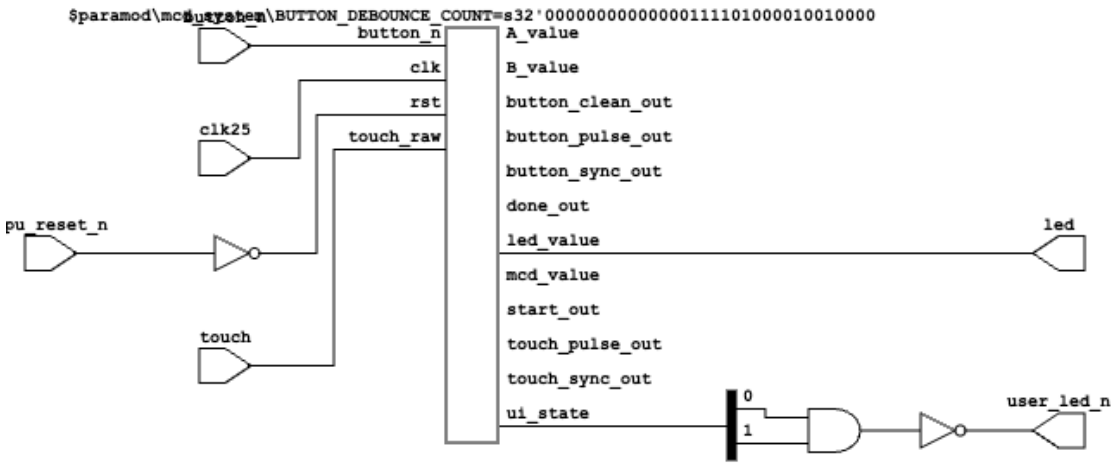


Figura 11: Vista del circuito sintetizado del sistema completo obtenida tras las etapas de optimización y mapeo lógico mediante Yosys.

La Figura 11 presenta una representación de menor nivel de abstracción, en la cual el diseño RTL ha sido transformado en elementos lógicos equivalentes mediante los procesos de optimización, *technology mapping* y sín-

tesis combinacional. Posteriormente, el diseño fue procesado mediante `nextpnr-ecp5` para realizar la ubicación y el enrutamiento de los recursos físicos de la FPGA. La frecuencia de reloj especificada para la implementación fue de 25 MHz.

Finalmente, el archivo de configuración generado fue cargado en la memoria SRAM de la FPGA utilizando `openFPGALoader` mediante la interfaz CMSIS-DAP. Durante la prueba física se verificó el ingreso de operandos mediante el panel táctil, el cambio entre las etapas de ingreso mediante el pulsador y la presentación binaria del resultado a través de ocho LED externos. Los resultados obtenidos en hardware coincidieron con los observados durante la simulación funcional.

Adicionalmente, con el propósito de inspeccionar con mayor profundidad la estructura interna y la interconexión detallada de cada submódulo, la lógica del sistema fue importada y analizada mediante el software Intel Quartus Prime. Esta herramienta permitió obtener una representación esquemática más detallada de los bloques lógicos componentes, facilitando la visualización del circuito sintetizado para el módulo principal (`mcd_system`), tal como se ilustra en la Figura 12.

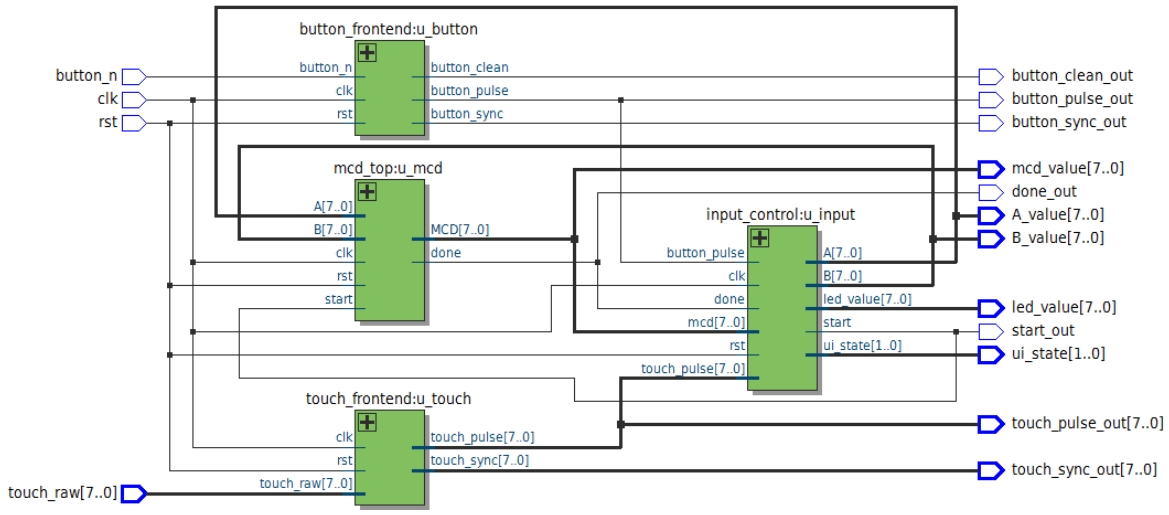


Figura 12: Circuito sintetizado del sistema completo obtenido en Quartus para el análisis detallado de sus componentes.

En la Figura 12 se pueden observar los cuatro submódulos principales que componen el sistema completo:

- `mcd_top` (`u_mcd`): implementa el algoritmo de Euclides para el cálculo del máximo común divisor. Recibe los operandos A y B, la señal de inicio `start` y genera el resultado `MCD[7..0]` junto con la señal de finalización `done`.
- `button_frontend` (`u_button`): realiza el acondicionamiento de la entrada física del pulsador (`button_n`). Se encarga de la sincronización de la señal con el reloj de la FPGA y del filtrado antirrebote (*debounce*), generando la señal limpia y detectora de flanco `button_pulse`.
- `touch_frontend` (`u_touch`): procesa las entradas asíncronas provenientes del panel táctil capacitivo `touch_raw[7..0]`. Realiza la sincronización de las 8 líneas de entrada con el dominio de reloj del sistema y detecta la pulsación de cada tecla, entregando el bus de pulsos detectados `touch_pulse[7..0]`.
- `input_control` (`u_input`): actúa como la unidad de interfaz y máquina de estados de usuario. Gestiona la secuencia de ingreso de datos coordinando las pulsaciones del teclado táctil y del botón para almacenar de forma iterativa los datos A y B, activar el cálculo mediante `start`, y multiplexar los valores mostrados en las salidas de los LED (`led_value[7..0]`) según el estado actual del sistema.

6.3.1. Submódulo `mcd_top`

El submódulo `mcd_top` representa la arquitectura jerárquica del núcleo de procesamiento matemático. La Figura 13 muestra la vista de alto nivel del circuito sintetizado, en la cual se distingue claramente la interacción

entre el camino de datos (`datapath`) y la unidad de control (`fsm`) a través de sus respectivas señales de control (`ldX`, `ldY`, `op`, `sel`) y de estado (`zero`, `neq`, `lt`).

Para profundizar en la implementación de hardware, se desglosa el módulo en niveles de menor abstracción obtenidos a partir del análisis en Intel Quartus Prime:

- **Camino de datos (`datapath`):** Ilustrado en la Figura 14, integra los registros de almacenamiento (`regX`, `regY`), multiplexores de selección, restador de 8 bits y lógica de comparación para procesar las operaciones aritméticas.
- **Unidad de control (`fsm`):** La Figura 15 muestra la estructura lógica a nivel de compuertas y bloques de la máquina de estados, encargada de decodificar el estado actual y generar las señales de habilitación.
- **Diagrama de transición de estados:** La Figura 16 expone la máquina de estados finita (FSM) sintetizada, donde se detallan los estados operativos del algoritmo (`S0_IDLE`, `S1_LOAD`, `S2_CHECK`, `S3_SUBY`, `S4_SUBX` y `S5_DONE`) y sus condiciones de transición.

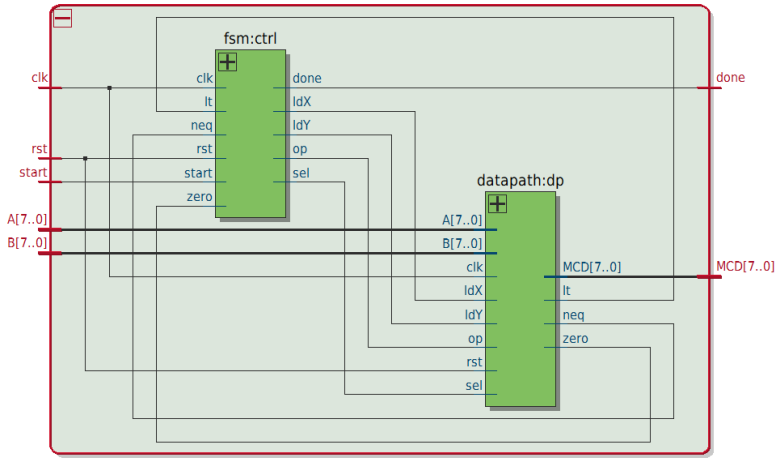


Figura 13: Circuito sintetizado del submódulo `mcd_top` e interconexión entre la FSM y el camino de datos.

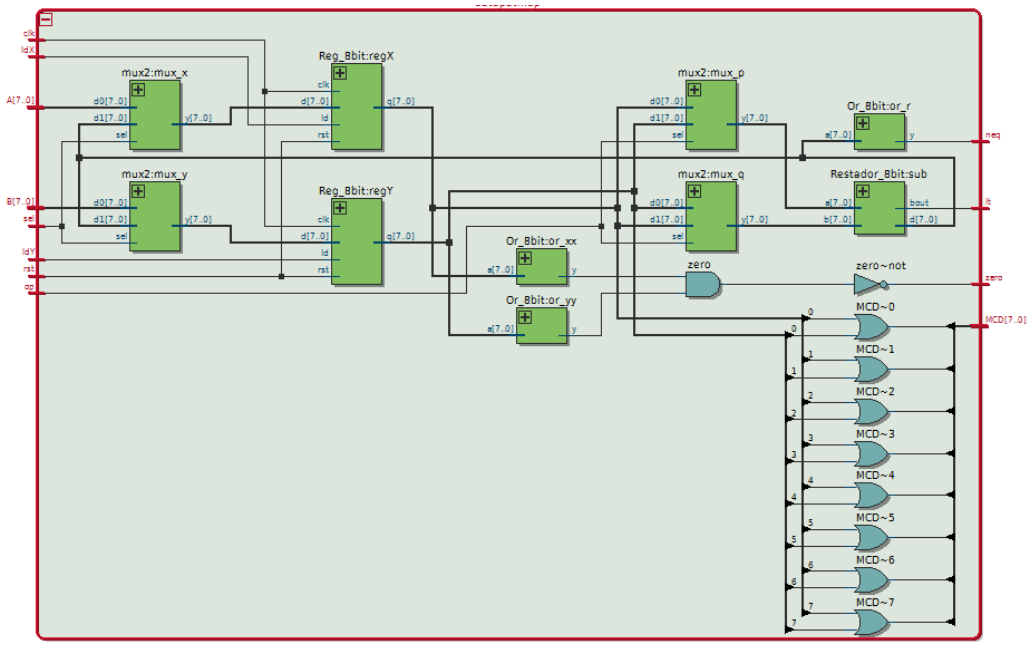


Figura 14: Estructura interna RTL del camino de datos (`datapath`).

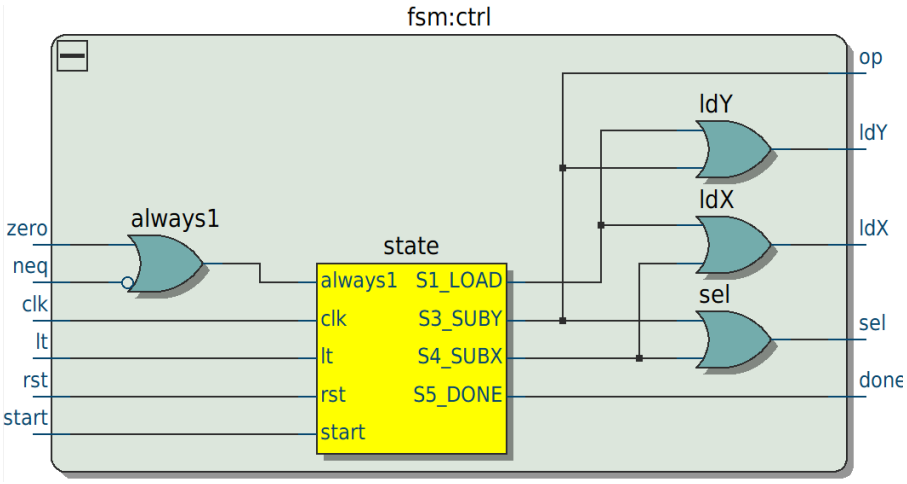


Figura 15: Esquemático de síntesis de la unidad de control (`fsm`).

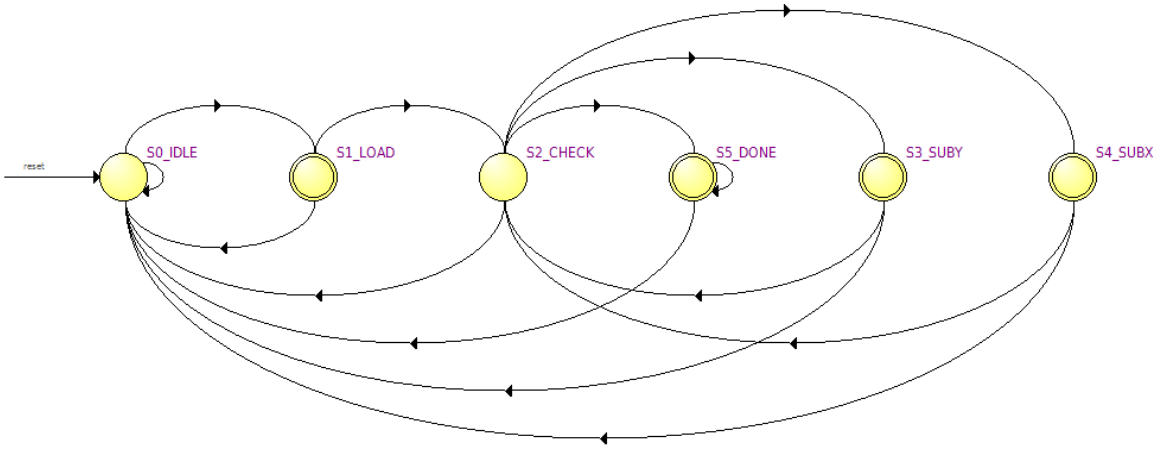


Figura 16: Diagrama de transición de estados generado por el sintetizador para la FSM del MCD.

6.3.2. Submódulos `button_frontend` y `touch_frontend`

Para garantizar la estabilidad en la captura de entradas de usuario y evitar estados metaestables o lecturas erróneas por rebotes mecánicos, el sistema incorpora dos etapas de acondicionamiento de entrada: `button_frontend` y `touch_frontend`.

El submódulo `button_frontend` (Figura 17) procesa la señal asíncrona del pulsador físico (`button_n`). Como se observa en su circuito sintetizado, la señal ingresa primero a una etapa de sincronización de dos flip-flops (`sync_2ff:u_sync`) para alinearse con el reloj del sistema. Posteriormente, la señal pasa por un bloque de filtrado antirrebote (`debounce:u_debounce`) que elimina los ruidos mecánicos del contacto, entregando una señal limpia (`button_clean`). Por último, la señal filtrada alimenta al módulo `edge_pulse:u_edge`, el cual detecta el flanco de bajada y genera un único pulso de un ciclo de reloj (`button_pulse`) por cada pulsación. Adicionalmente, el submódulo ofrece la señal sincronizada e invertida (`button_sync`).

Por otro lado, el submódulo `touch_frontend` (Figura 18) se encarga de acondicionar el bus de 8 líneas del teclado táctil capacitivo (`touch_raw[7..0]`). Al ser una interfaz multicanal, el bus pasa primero de forma paralela por el sincronizador `sync_2ff:u_sync` para generar `touch_sync[7..0]`. A diferencia del pulsador mecánico, las teclas táctiles no requieren una etapa de *debounce*; por ello, la salida sincronizada pasa directamente al detector de flancos `edge_pulse:u_edge`, generando el bus de pulsos `touch_pulse[7..0]`.

A nivel interno, los bloques reutilizables que componen estas etapas se implementan de la siguiente manera:

- **Sincronizador (`sync_2ff`):** Ilustrado en la Figura 19, consiste en una cadena en cascada de dos flip-flops tipo D (`sync_ff1` y `sync_ff2`) con multiplexores de reinicio asíncrono, diseñada para mitigar la

metaestabilidad en las entradas externas.

- **Filtro antirrebote (`debounce`):** Mostrado en la Figura 20, utiliza un contador de 18 bits (`counter[17..0]`) acoplado a un comparador (`Equal0`) y lógica de estado para validar que la señal mantenga un nivel estable durante un tiempo determinado antes de actualizar la salida `signal_out`.
- **Detector de flanco (`edge_pulse`):** Representado en la Figura 21, emplea un registro flip-flop (`previous`) para almacenar el estado anterior de la señal de entrada e interconecta su salida invertida con una compuerta AND, generando un pulso activo en la salida `pulse_out` únicamente en el ciclo de reloj en que ocurre el flanco de subida.

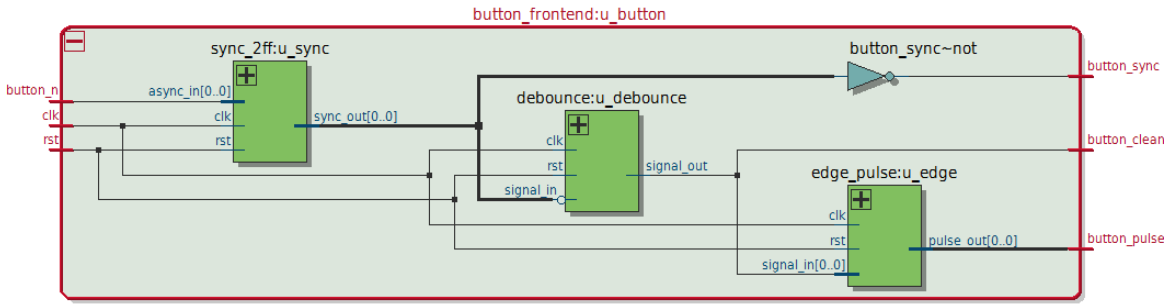


Figura 17: Circuito sintetizado del submódulo `button_frontend`.

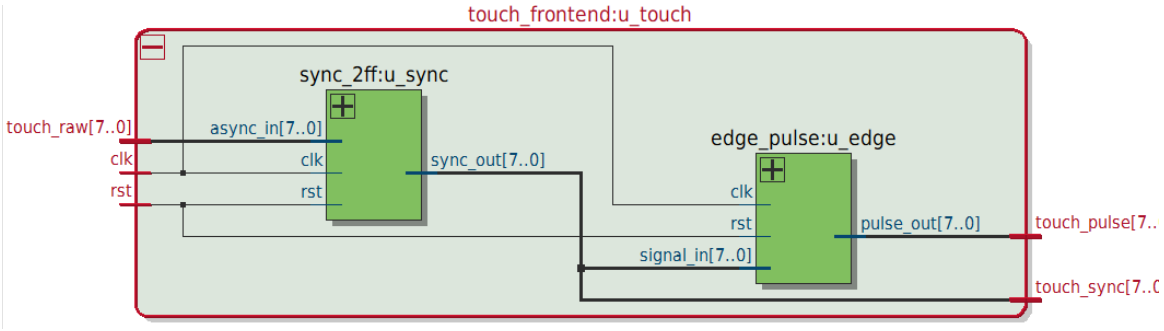


Figura 18: Circuito sintetizado del submódulo `touch_frontend`.

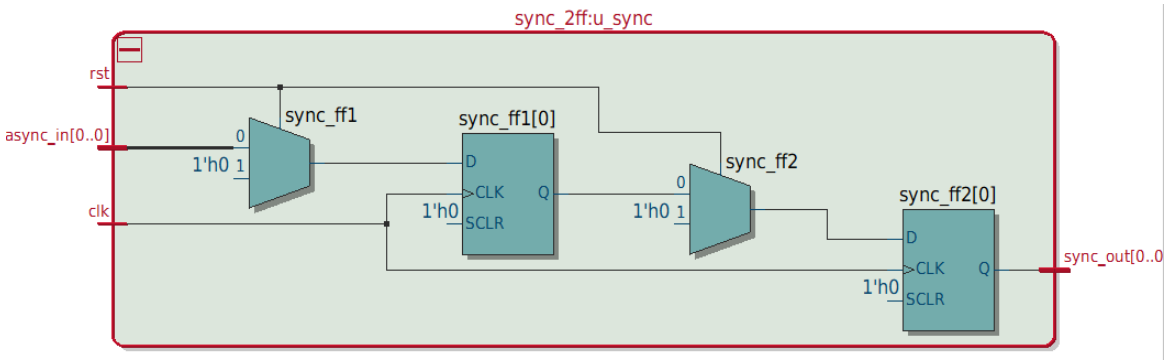


Figura 19: Esquemático RTL del sincronizador de dos flip-flops (`sync_2ff`).

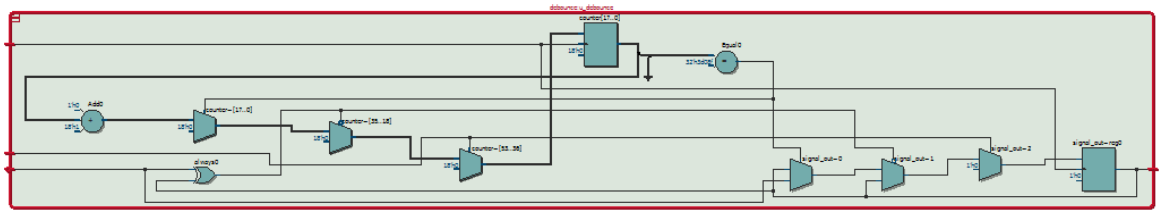


Figura 20: Esquemático RTL del módulo de filtrado antirrebote (`debounce`).

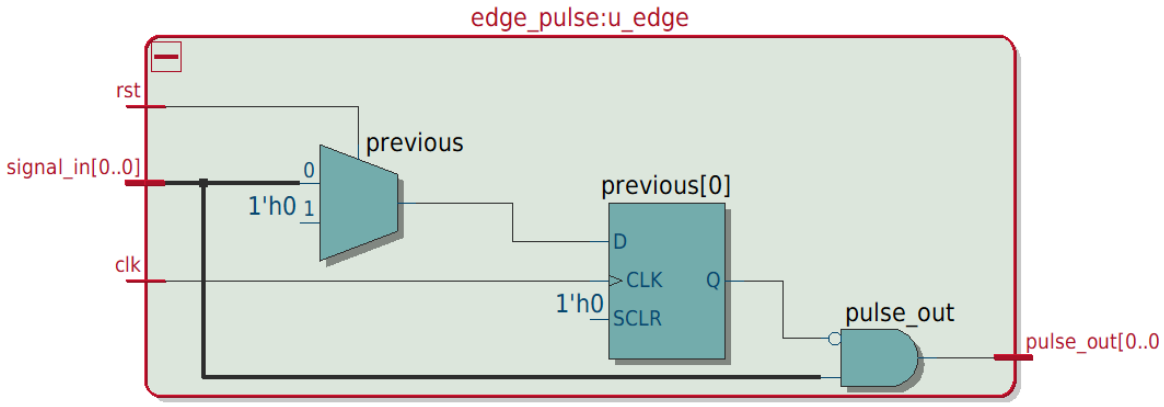


Figura 21: Esquemático RTL del detector de flancos (`edge_pulse`).

6.3.3. Submódulo `input_control`

El submódulo `input_control` actúa como la unidad de interfaz y máquina de estados del usuario, coordinando la secuencia de captura de operandos, la activación del núcleo computacional y la selección de la información visualizada en las salidas del sistema.

Como se observa en el esquemático de síntesis (Figura 22), el módulo integra la lógica de control, registros de estado y multiplexación de salidas:

- **Gestión de la FSM de usuario (`ui_state`):** Administra las etapas del flujo de operación (`LOAD_A`, `LOAD_B`, `CALCULATE` y `DONE`). Utiliza registros de estado (`ui_state[1..0]`) y comparadores lógicos para conmutar progresivamente con cada pulso recibido del botón (`button_pulse`).
- **Captura y almacenamiento de operandos:** Procesa los impulsos del teclado táctil (`touch_pulse[7..0]`) mediante arreglos de compuertas lógicas y multiplexores de selección (`Mux0`–`Mux23`). Estos se encargan de decodificar la tecla presionada y cargar el valor correspondiente en los registros de almacenamiento de los operandos A y B (`A[7..0]` y `B[7..0]`).
- **Control del cálculo (`start`):** Al transicionar al estado de cálculo, la lógica genera un pulso sincronizado en la señal `start` para iniciar la ejecución del algoritmo de Euclides en el módulo `mcd_top`.
- **Multiplexación de la salida visual (`led_value`):** Selecciona dinámicamente el bus de datos presentado en las salidas de los LED según el estado de la interfaz (`ui_state`). Dependiendo de la fase actual, dirige hacia `led_value[7..0]` el valor almacenado para A, el valor cargado para B o el resultado final del máximo común divisor (`mcd[7..0]`) una vez completado el procesamiento.

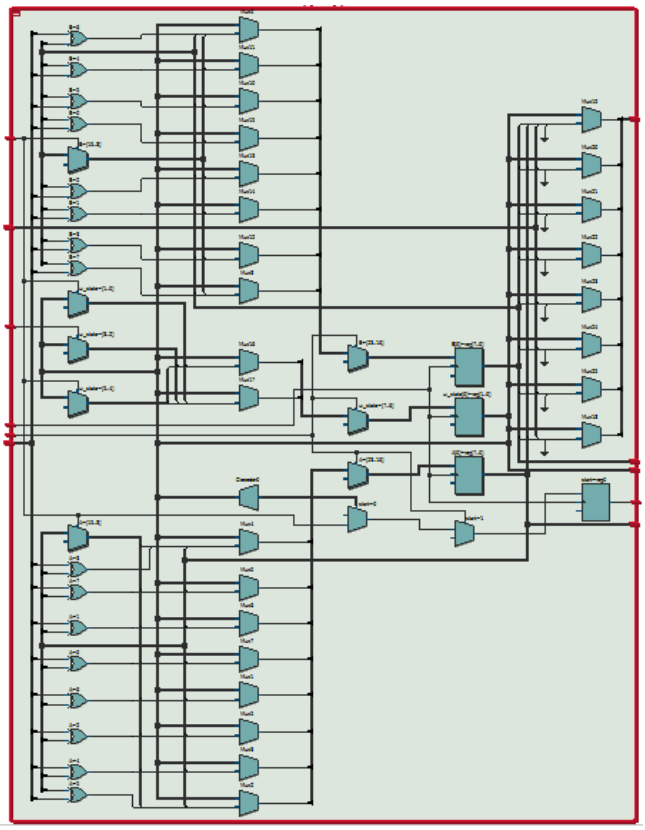


Figura 22: Circuito sintetizado del submódulo `input_control`, mostrando los arreglos de multiplexores, registros de almacenamiento y lógica de control de la interfaz.

6.4. Recursos utilizados

Una vez finalizado el proceso de síntesis, *packing*, ubicación y enrutamiento, se analizó la cantidad de recursos físicos empleados por el diseño sobre la FPGA Lattice ECP5 LFE5U-45F.

La herramienta Yosys reportó, para la descripción sintetizada, 132 bloques `LUT4`, 89 flip-flops `TRELLIS_FF`, 9 bloques `CCU2C`, 16 multiplexores `PFUMX` y 3 multiplexores `L6MUX21`. Después de la etapa de *packing*, `nextpnr` reportó un total de 150 LUT4 utilizadas, correspondientes a 132 LUT destinadas a lógica general y 18 LUT asociadas a las cadenas de acarreo. Estos valores son considerablemente inferiores a la capacidad disponible en el dispositivo.

Esos valores salen directamente de la síntesis: 132 LUT4 y 89 FF en Yosys, y nextpnr reportó 150 LUT totales —132 lógicas y 18 de carry— sobre 43.848 disponibles.

Cuadro 5: Utilización de recursos de la FPGA Lattice ECP5 LFE5U-45F.

Recurso	**Utilizados**	**Disponibles**	**Ocupación**
LUT4 totales	150	43848	0.342 %
LUT4 de lógica	132	43848	0.301 %
LUT4 de acarreo	18	43848	0.041 %
Flip-Flops	89	43848	0.203 %
Entradas/Salidas	20	245	8.163 %
Red global de reloj	1	56	1.786 %
Memoria BRAM	0	108	0 %
Bloques DSP	0	72	0 %
PLL	0	4	0 %

El bajo porcentaje de utilización evidencia que la arquitectura propuesta es compacta respecto a la capacidad total del dispositivo. En particular, la reutilización de un único restador para ejecutar tanto $X - Y$ como $Y - X$ permite evitar la duplicación innecesaria de lógica aritmética. El recurso con mayor porcentaje de ocupación corresponde a los bloques de entrada/salida, debido a que el sistema utiliza buses independientes para las ocho entradas del panel táctil y las ocho salidas hacia el arreglo de LED, además de las señales de reloj, reinicio y pulsador. En cuanto al desempeño temporal, después del proceso completo de *place and route*, `nextpnr-ecp5` determinó una frecuencia máxima estimada de 165,29 MHz. Debido a que la frecuencia de operación establecida para el sistema es únicamente de 25 MHz, el diseño cumple ampliamente con la restricción temporal impuesta. El período correspondiente a la máxima frecuencia estimada es aproximadamente:

$$T_{\text{crit}} = \frac{1}{165{,}29 \times 10^6} \approx 6{,}05\,\text{ns} \tag{1}$$

mientras que el período de operación real del sistema es:

$$T_{\text{clk}} = \frac{1}{25 \times 10^6} = 40\,\text{ns}. \tag{2}$$

Por lo tanto, existe un margen temporal considerable entre el camino crítico del circuito y el período de reloj utilizado. El reporte final de `nextpnr` confirmó el cumplimiento de temporización (`PASS at 25.00 MHz`).

7. Anexos

Todo el código, diagramas e implementación está disponible públicamente en el repositorio de github o en la url a continuación: `https://github.com/davidvillarreal2901/MCD_FPGA.git`.

Referencias

[1] *Eslava Garzón, J. S. (2026). Práctica 01: Metodología de Diseño Tradicional. En Verificación de Sistemas Digitales (2026-02). Departamento de Ingeniería Eléctrica y Electrónica, Universidad Nacional de Colombia, Sede Bogotá.*